Onboard Deep Lossless and Near-Lossless Predictive Coding of Hyperspectral Images With Line-Based Attention

Diego Valsesia, *Member, IEEE*, Tiziano Bianchi, *Member, IEEE*, and Enrico Magli, *Fellow, IEEE*

***Abstract*— Deep learning methods have traditionally been difficult to apply to compression of hyperspectral images onboard spacecrafts due to the large computational complexity needed to achieve adequate representational power, as well as the lack of suitable datasets for training and testing. In this article, we depart from the traditional autoencoder approach, and we design a predictive neural network, called line receptance weighted key value (LineRWKV), which works recursively line by line to limit memory consumption. In order to achieve that, we adopt a novel hybrid attentive-recursive operation that combines the representational advantages of Transformers with the linear complexity and recursive implementation of recurrent neural networks (RNNs). The compression algorithm performs the prediction of each pixel using LineRWKV, followed by entropy coding of the residual. Experiments on multiple datasets show that LineRWKV is highly memory-efficient, significantly outperforms state-of-the-art deep learning methods, and is the first deep learning approach to outperform CCSDS-123.0-B-2 at lossless and near-lossless compression. Promising throughput results are also evaluated on a 7-W embedded system.**

***Index Terms*— Deep learning, hyperspectral image compression, predictive coding, receptance weighted key value (RWKV), self-attention.**

I. Introduction

HYPERSPECTRAL images acquired by spacecrafts are an essential tool for Earth observation, playing pivotal roles in environmental monitoring, urban planning, tackling climate change, and much more [1], [2], [3], [4], [5], [6]. The spatial and spectral resolution of instruments keeps growing to satisfy the demands of final users. However, this poses significant challenges for the transmission and management of huge amounts of spatial and spectral information, making compression a topic of paramount importance. Even modest gains in rate-distortion performance may result in significant bandwidth reduction. This is critical for many tasks where downlink bandwidth is a limiting factor in the timeliness of the delivery of the products or their resolution [7]. The challenge of onboard compression of hyperspectral images is far from trivial, as it requires methods that are efficient in processing massive amounts of data with the limited computational resources that are available on a satellite and, at the same time, are able to capture complex spatial–spectral redundancy patterns.

Deep learning has shown great promise for the task of RGB image and video compression [8], [9], [10], [11], [12], [13]. However, the literature on deep learning for hyperspectral image compression, particularly for usage onboard of spacecrafts, is still in its infancy due to the complexity of this data type. In fact, effective compression of hyperspectral images requires capturing both spatial and spectral correlation patterns. When a neural network is to be designed for this task, it requires processing of the entire 3-D data cube to extract spatiospectral features that are relevant to the compression task. However, in doing so, complexity can grow very rapidly both in the amount of operations to be performed by neural layers and, even more so, in memory requirements. This is clearly at odds with the limited computational resources available onboard.

The literature on deep hyperspectral image compression is currently focused on designs based on autoencoders, following seminal works for RGB images [8]. In this kind of approach, the neural network generally consists of an encoder, which maps the image into a compact latent space, and a decoder, which reconstructs the image. Several designs are possible depending on the focus on spatial or spectral features, complexity tradeoffs, and so on. For instance, some works [14], [15] limit complexity by just focusing on spectral redundancy, while others seek complex spatiospectral representations [16], [17], [18], [19], [20] with various operations and training objectives to enhance the quality of the features. Several of these works do not attempt to carefully control complexity for onboard usage. The work by Verdú et al. [21] started posing the question of a model with a complexity that would be suitable for onboard usage and presented a design accordingly.

Common to the entire existing literature on deep hyperspectral image compression is the low-bit-rate setting in which all these autoencoder methods are tested. Results are typically presented at rates lower than 1 bit per pixel per channel (bpppc), corresponding to very high compression ratios, and quality levels that would generally be unacceptable for real remote sensing missions. This is the regime where such methods are most competitive due to an intrinsic limitation of the autoencoder design. Indeed, reconstructing an image from a latent space with very high fidelity requires neural networks

Received 26 March 2024; revised 18 July 2024 and 27 August 2024; accepted 18 September 2024. Date of publication 20 September 2024; date of current version 7 October 2024. This work was supported by the European Union Next-GenerationEU (PIANO NAZIONALE DI RIPRESA E RESILIENZA (PNRR)—MISSIONE 4 COMPONENTE 2, INVESTIMENTO 1.3—D.D. 1555 11/10/2022, PE00000013). *(Corresponding author: Diego Valsesia.)*

The authors are with the Department of Electronics and Telecommunications, Politecnico di Torino, 10129 Turin, Italy (e-mail: diego.valsesia@polito.it).

Digital Object Identifier 10.1109/TGRS.2024.3465043

1558-0644 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

with large representational power, both in the encoder and decoder parts. Due to this, existing designs do not scale well to a high-rate, high-quality regime, presenting a plateau in rate-distortion performance as the rate is increased or exploding complexity. In fact, as we show in this article, the state of the art in deep learning compression of hyperspectral images is significantly outperformed by classic approaches, such as the low-complexity CCSDS-123.0-B-2 standard [22] for onboard lossless and near-lossless compression, when targeting the high-quality regime. The lack of lossless compression methods is another limitation of the current literature that we address in this work. Hence, it is clear that in order to address the weaknesses of the existing literature in the high-quality regime, both in terms of rate-distortion performance and low-complexity designs, we need to depart from the autoencoder approach.

In this article, we present a novel neural network design, called line receptance weighted key value (LineRWKV), capable of lossless and lossy compression. The design departs from existing works in several ways. First, it follows a predictive coding approach, where the neural network predicts a pixel value from a causal context and only the prediction residual is entropy-coded. This is to address the aforementioned limitation of autoencoders where reconstruction from the bottleneck is hard to scale with limited complexity since a predictor can be simpler to design. This also allows us to readily target lossless compression, which is currently neglected by the literature, as well as lossy compression with a bounded error (so-called near-lossless). Moreover, the proposed architecture has a radical novel design to constrain complexity, particularly regarding memory usage. We propose a neural network architecture that can work recursively in the along-track direction. This limits memory usage, as it is only tied to one image line with all its spectral channels while exploiting a memory of past lines and enabling continuous pushbroom operation. Critically, we use a novel hybrid attentive-recursive operation [23], which avoids pitfalls of older recurrent neural networks (RNNs), enabling parallelized training and having the representational power of Transformers [24], with the advantage of linear complexity instead of quadratic. While [23] has been proposed for natural language processing tasks, we are the first to adapt this architecture to image processing. We conduct extensive experiments on three datasets to show that for the first time for a deep-learning method, we can significantly outperform CCSDS-123.0-B-2 in rate-distortion performance. We also conduct tests on a 7-W embedded system to validate promising throughput results, matching our low-complexity design goals.

In summary, the main contributions presented in this article are given as follows:

1) the first scheme for deep hyperspectral image compression departing from autoencoders by adopting predictive coding;
2) a novel low-complexity architecture that constrains memory and computational requirements by working line by line with a novel hybrid-recurrent scheme;
3) the first deep hyperspectral image compression for lossless compression;
4) the first deep learning method to outperform CCSDS-123.0-B-2 in the high-rate, high-quality regime.

A preliminary version of this work [25] introduced the general idea, while this article significantly expands the methodology and experimental results. The manuscript is organized as follows. Section II introduces some background material on methods for compression of remote sensing hyperspectral images, both traditional and based on deep learning, as well as background on deep sequence processing which is useful to understand the proposed method. Section III presents the technical description of the proposed method. Section IV reports compression results on multiple datasets. Finally, Section V draws some conclusions and discusses the main limitations of the method for future improvement.

II. Background

A. Compression Onboard of Spacecrafts

Onboard hyperspectral image compression faces a challenging balance between rate-distortion performance and computational complexity. Over the years, CCSDS standards have gained popularity due to their low computational demands, enabling high throughput on dedicated FPGAs [26], and good rate-distortion performance. In particular, CCSDS 122.0-B-2 [27] together with CCSDS 122.1-B-1 [28] employs a transform coding approach to lossless and lossy compression, where a 2-D discrete wavelet transform is coupled with a spectral transform. The more recent CCSDS 123.0-B-2 [22] follows, instead, a predictive coding approach to lossless and near-lossless (i.e., lossy with bounded error) compression with a spatial–spectral predictor based on an adaptive filter. CCSDS 123.0-B-2 is regarded as the state-of-the-art for onboard hyperspectral image compression and, to the best of our knowledge, remains unchallenged by current deep-learning methods. Finally, it is worth mentioning that regression wavelet analysis (RWA) [29], [30] combines a spectral transform with 2-D JPEG2000 to achieve compression performance competitive with CCSDS 123.0-B-2.

B. Image Compression With Deep Learning

Most of the works in the literature on image compression with deep learning stem from the computer vision community, focusing on 8-bit RGB or gray-scale images [8], [9], [10], [11], [12], [13].

Few works [14], [15], [16], [17], [18], [19], [20], [21], [31] have addressed the topic of hyperspectral images, especially while keeping complexity in mind for possible onboard usage. The general approach mainly follows the early work of Ballé et al. [8], in the design of autoencoder neural networks where an encoder subnetwork creates a latent representation of the input image in a low-dimensional space, which is then quantized and entropy-coded. Sometimes, encoder features are also used to derive a hyperprior on the distribution of the low-dimensional latent representation. A decoder subnetwork maps the latent representation back to the image space. These networks are trained end to end with rate-distortion objectives, balancing, according to the desired tradeoff, the reconstruction

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

error and the entropy of the latent representation. Recent advancements for RGB images focused on improving encoder architectures and context models for the entropy coder [12]. Regarding hyperspectral images, the main challenge lies in the extremely high memory requirements to capture 3-D spatial–spectral features for a compressed latent representation. The works by Kuester et al. [14] and La Grassa et al. [15] focus on designing a spectral autoencoder. This limits the computational complexity and captures a significant amount of correlation, as it is known that hyperspectral images are more spectrally than spatially correlated, but does not exploit spatial redundancy, which might be significant. Most other works fully exploit spatiospectral features, e.g., via the simple convolutional autoencoders designed in [16] and [31]. Guo et al. [18] propose a variational autoencoder with an edge extractor to provide guidance to better represent edges. In subsequent work, Guo et al. [19] use cross-channel contrastive learning to enhance spectral features but do not attempt to optimize complexity or memory usage. In a recent work targeting onboard compression, Verdú et al. [21] adopt a channel clusterization strategy and novel normalization strategies needed for images with high dynamic range. While all these methods based on autoencoders work well at low rates, typically being tested below 1 bpppc, they are difficult to scale to high-rate, high-quality regimes without compromising efficiency. In fact, the autoencoding approach tends to struggle when very low distortion is desired if the network capacity in terms of the number of features, latent space size, and the total number of parameters is not adequately scaled.

Finally, it is worth mentioning that some autoregressive approaches [32], [33] have been studied on RGB images. They generally rely on modeling the image as a sequence of all pixels and use a causal context for prediction within the mechanism of either causal convolution, RNNs, or Transformers. As detailed in Section II-C, until recently, each of these approaches had a critical limitation, consisting of either limited representational power, inefficient training and encoding, or high complexity. We also remark that the proposed method is not purely autoregressive on all image pixels since the across-track dimension can be effectively processed in parallel, as detailed in Section III-G.

C. Deep Sequence Processing

Since the proposed method is based on a causal prediction approach, it is worth reviewing the main ways in which deep neural networks are used to process sequences and their respective tradeoffs. For simplicity, we are going to discuss the processing of a 1-D sequence.

First, a simple design would be causal convolution, such as that of PixelCNN [34], where the convolution kernel is masked so that the receptive field only expands in the past samples. While this approach enjoys fairly efficient implementations, it has limited representational power as it is not an input-dependent operation (like attention is), and the size of the receptive field, i.e., how far past samples affect the current prediction, might be limited.

An alternative approach would be the use of RNNs, such as the long short-term memory (LSTM) [35], to have ideally infinite memory. This approach is limited by the inefficiency of training, which requires serial computations over the entire sequence length, so that it does not scale to the large datasets required to train even moderately complex models.

Fig. 1. Overview of compression scheme for near-lossless compression. A raw image is prequantized by means of a uniform scalar quantizer and then fed to a causal prediction model implemented by the LineRWKV neural network, producing prediction residuals that are entropy-coded.

Transformers [24] are currently the state of the art for sequence processing, thanks to the high representational power of the input-dependent attention operation, which is capable of creating operations that are adaptive to the sequence under processing. Unlike RNNs, Transformers can also be parallelized during training. However, they suffer from quadratic complexity in the sequence length and require keeping the entire sequence and its features in memory, resulting in high memory and computational costs. This renders them infeasible to process an image in an autoregressive pixelwise fashion.

Very recently, the natural language processing literature has focused its attention on sequence processing with hybrid designs between RNNs and Transformers, which simultaneously have linear complexity, admit parallel training, and can be written in a recursive manner for low-memory inference [23], [36]. Receptance weighted key value (RWKV) [23] is a recent model with such properties, which is at the basis of the proposed work, as detailed in Section III.

III. Proposed Method

A. Overview

In this section, we present the full design of the proposed hyperspectral compression method, as shown in Fig. 1. This is a predictive coding method, where a pixel is predicted from a causal spatial and spectral context, and only the prediction error is encoded. The method comprises a prequantizer to introduce losses, followed by a deep-learning-based predictor, called LineRWKV, and an entropy encoder to code the prediction residuals.

Regarding the prequantizer module, lossy compression, in the context of predictive coding algorithms, can be achieved in two ways: in-loop quantization of prediction errors or image prequantization. In-loop quantization predicts a pixel value, computes a prediction residual, and quantizes it. However, in order to complete the prediction for the next pixel value, it is necessary to reconstruct the pixel value from the quantized residual. This is to ensure that the decoder can perform the same prediction operation. However, it introduces serial data dependencies. On the other hand, prequantization consists of quantizing the input image and then using lossless prediction. This avoids data dependencies at the compressor since there is no discrepancy between the original pixel value and the value to be used for prediction, so it leads to higher compression throughput. While in-loop quantization is theoretically superior, this leads to inefficient implementations, and at the high rates desirable for usage in real missions, which are the focus

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

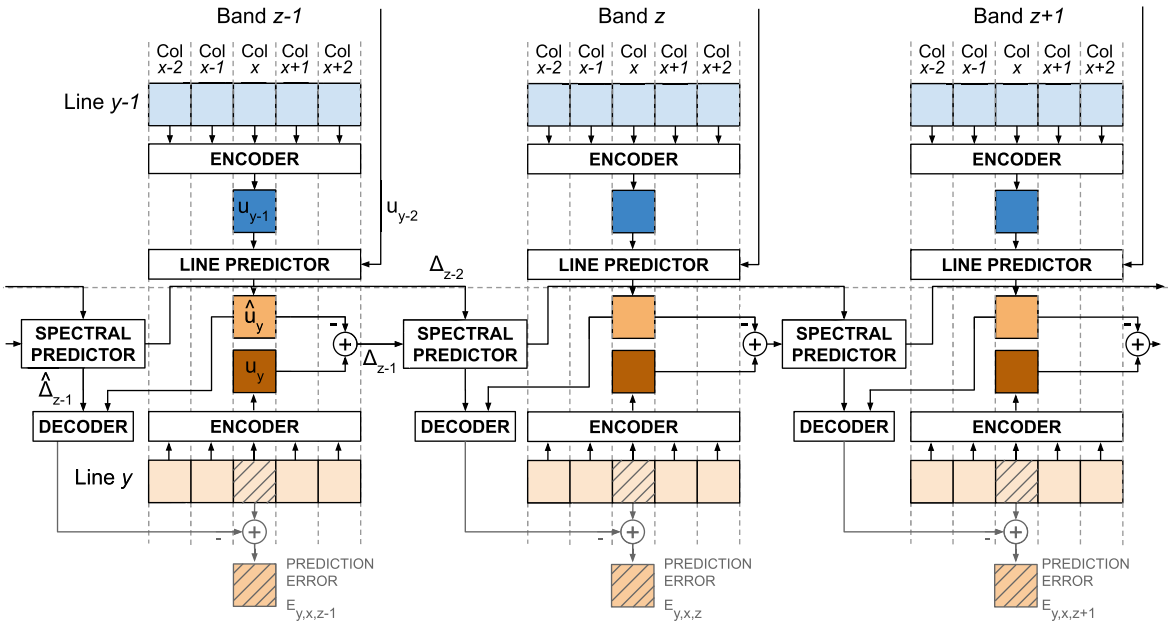

Fig. 2. Overview of the LineRWKV predictor for $z > 1$ and $y > 0$. Encoder, decoder, line, and spectral predictors are neural networks trained end to end to minimize the prediction error for all pixels. Each square represents a pixel in the corresponding spatial–spectral position, as it is processed by the neural modules. Rounded prediction errors are then entropy-coded with a standard code.

of this article, the rate-distortion penalty of prequantization with respect to in-loop quantization is minimal, as shown in [37]. Therefore, this work adopts a prequantization block using a uniform scalar quantizer with odd-integer step size Q to obtain the prequantized image $I^q = \text{round}(I/Q)$.

The entropy encoder module can use any method with suitable complexity. In this work, we use the sample-adaptive Golomb encoder defined by the CCSDS-123.0-B-2 standard [22] as entropy coder for the positive-mapped prediction residuals to ensure fair comparisons with the CCSDS predictor.

LineRWKV is a neural network implementing a predictor, i.e., it estimates the value of a pixel based on a context (network receptive field) of past spatial and spectral pixels. For clarity of explanation, we will describe its behavior in inference mode for the compression stage. Discrepancies with respect to the training process are discussed in Section III-G. A high-level overview of LineRWKV is presented in Fig. 2, and Algorithm 1 recaps the operations performed by the modules presented in Sections III-B–III-F. We denote sampling of the along-track direction of the satellite with the term "lines" and variable $y = 0, 1, 2, \ldots$; the across-track direction is denoted as columns and variable $x = 0, 1, 2, \ldots, N_x - 1$; and the spectral direction is denoted as $z = 0, 1, 2, \ldots, N_z - 1$. LineRWKV does not pose any restrictions on the extent of the three dimensions, which may be different between training and testing. In particular, the maximum number of columns and bands is dictated by available memory, while the number of lines can be infinite, supporting continuous pushbroom acquisition. The prediction for pixel $I^q_{y=Y,x=X,z=Z}$ depends on all pixels with $y \leq Y$ and $z < Z$ $\forall x$. We note that the context used by LineRWKV for prediction is slightly different from conventional nonneural designs. Specifically, the predictor employs all lines above the current pixel and all previous bands for the current pixel, but it does not use information from the previous pixels on the same line or future bands in past lines. This is due to the LineRWKV design, which employs separate blocks in the spectral, along-track, and across-track dimensions. This choice is slightly suboptimal; it is similar in principle to the "narrow" mode of the CCSDS-123.0-B-2 predictor, which incurs a small loss of compression efficiency [37], but it is necessary in order to avoid a much more complex structure of the neural predictor. The predicted value is rounded and subtracted from the original pixel value to compute the prediction error. Finally, the prediction error is losslessly encoded with any entropy encoder. At any time during inference, the last two lines of the image with all their spectral channels (i.e., a tensor of size $2 \times N_x \times N_z$) are supposed to be under processing to predict the next line, which is also available for the computation of the residual.

Algorithm 1 LineRWKV Predictor

Require: Prequantized image $\mathbf{I}^q = \text{round}(\mathbf{I}/Q)$
Require: Init $\mathbf{u}_{-1,x,z} = \mathbf{0}$ $\forall x, z$
Require: Init states $\mathbf{a}_{-1,x,z}, \mathbf{b}_{-1,x,z} = \mathbf{0}$ $\forall x, z$
Compress first line (Section III-F) to get $E_{0,x,z}$ $\forall x, z$
for $y = 1, \ldots$ **do**
$\mathbf{u}_{y-1,x,z} \leftarrow \text{Encoder}(\mathbf{I}^q_{y-1,:,z})$ $\forall x, z$
$\hat{\mathbf{u}}_{y,x,z}, (\mathbf{a}, \mathbf{b})_{y-1,x,z} \leftarrow \text{LinePredictor}(\mathbf{u}_{y-1,x,z}, \mathbf{u}_{y-2,x,z}, \mathbf{a}_{y-2,x,z}, \mathbf{b}_{y-2,x,z})$ $\forall x, z$
$E_{y,x,0} = \text{Decoder}_0(\hat{\mathbf{u}}_{y,x,0}) - I^q_{y,x,0}$ $\forall x$
$\boldsymbol{\Delta}_{y,x,z-1} = \mathbf{u}_{y,x,z-1} - \hat{\mathbf{u}}_{y,x,z-1}$ $\forall x, z \geq 1$
$\hat{\boldsymbol{\Delta}}_{y,x,z} = \text{SpectralPredictor}(\{\boldsymbol{\Delta}_{y,x,\zeta}\}_{\zeta<z})$ $\forall x, z \geq 1$
$\hat{I}^q_{y,x,z} = \text{Decoder}(\hat{\boldsymbol{\Delta}}_{y,x,z} + \hat{\mathbf{u}}_{y,x,z})$ $\forall x, z \geq 1$
$E_{y,x,z} = \hat{I}^q_{y,x,z} - I^q_{y,x,z}$ $\forall x, z \geq 1$
end for

The LineRWKV predictor has a modular architecture design composed of the following blocks: 1) an *encoder* to map each pixel in a line and given band into a feature space by exploiting across-track spatial correlation; 2) a *line predictor*

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

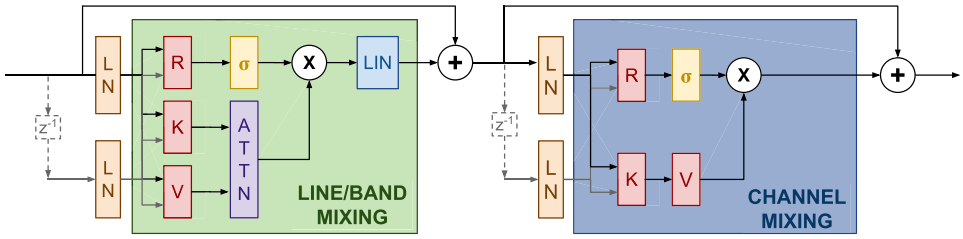

Fig. 3. Line and spectral predictors are made of the repetition of line/band mixers and channel mixers according to the depicted architecture. z^{-1} denotes the unit delay in the sequence dimension (lines or bands). LN denotes the LayerNorm, and LIN denotes the linear projection of features.

that predicts the features of the colocated (same x, z) pixel in the next line; 3) a *spectral predictor* that predicts the features of the colocated (same x, y) pixel in the next band; and 4) a *decoder* that estimates a pixel value based on its features. Sections III-B–III-F explain each of the blocks in detail. In the following, we suppose that we are predicting the values for line y from current line $y-1$ and past line $y-2$, as depicted in Fig. 2.

B. Encoder

The goal of the encoder is to capture the correlation that exists across image columns and encode it into a feature space for further processing. The encoder function is, thus, a 1-D operation that is shared for all bands and for all lines. A simple design for the encoder is a sequence of blocks composed of 1-D convolution, layer normalization [38], and nonlinear activation. This design should be modulated according to the expected complexity of the across-track correlation patterns, possibly considering larger receptive fields and attention operations, if complexity allows. For our experiments, we choose the simple convolutional blocks, and we denote the number of such blocks with N_{enc}. The encoding of the input line will result in a feature vector $\mathbf{u}$ for each spatial–spectral pixel (i.e., a tensor of size $1 \times N_x \times N_z \times F$).

C. Line Predictor

The line predictor is the core operation of LineRWKV, which enables the recursion over the image lines, constraining memory usage and enabling continuous operation. The line predictor should exploit the features of a number of past lines in order to predict the next line. As discussed in Section II-C, it is desirable to have an attention-based operation such as the scaled dot-product attention of the Transformer to process the sequence of lines, both to parallelize training and to exploit the power of attention to adapt to self-similar patterns. However, Transformers require keeping several lines and their features in memory, which is prohibitive, and have quadratic computational cost in the number of lines, which is also prohibitive. On the other hand, training a traditional RNN on a large scale would be extremely slow and would not scale well. Our line predictor is therefore based on the RWKV neural network [23] for sequence processing, and its recurrent implementation is used for inference.

More in detail, each of the $N_x N_z$ pixels in the current line, represented as a feature vector by the across-track encoder, is processed in parallel by the line predictor. The RWKV-based line predictor is composed of the repetition of two fundamental blocks: line mixing and channel mixing, both depicted in Fig. 3. Overall, N_{lp} line and channel mixing blocks are used in our architecture. For brevity of notation, we denote the feature vector of a pixel input to the line mixing operation as $\mathbf{u}_{y-1} \in \mathbb{R}^F$ and its corresponding output as $\mathbf{o}_{y-1} \in \mathbb{R}^F$, omitting subscripts x, z. In particular, line mixing performs the following operations:

$$\mathbf{r}_{y-1} = \mathbf{W}_r(\mu_r \mathbf{u}_{y-1} + (1-\mu_r)\mathbf{u}_{y-2}) \tag{1}$$
$$\mathbf{k}_{y-1} = \mathbf{W}_k(\mu_k \mathbf{u}_{y-1} + (1-\mu_k)\mathbf{u}_{y-2}) \tag{2}$$
$$\mathbf{v}_{y-1} = \mathbf{W}_v(\mu_v \mathbf{u}_{y-1} + (1-\mu_v)\mathbf{u}_{y-2}) \tag{3}$$
$$\mathbf{a}_{y-1} = e^{-\boldsymbol{\alpha}} \odot \mathbf{a}_{y-2} + e^{\mathbf{k}_{y-1}} \odot \mathbf{v}_{y-1} \tag{4}$$
$$\mathbf{b}_{y-1} = e^{-\boldsymbol{\alpha}} \odot \mathbf{b}_{y-2} + e^{\mathbf{k}_{y-1}} \tag{5}$$
$$\mathbf{p}_{y-1} = \frac{\mathbf{a}_{y-2} + e^{\boldsymbol{\beta}+\mathbf{k}_{y-1}} \odot \mathbf{v}_{y-1}}{\mathbf{b}_{y-2} + e^{\boldsymbol{\beta}+\mathbf{k}_{y-1}}} \tag{6}$$
$$\mathbf{o}_{y-1} = \mathbf{W}_o(\sigma(\mathbf{r}_{y-1}) \odot \mathbf{p}_{y-1}) \tag{7}$$

where $\mathbf{W}_*$, $\boldsymbol{\alpha}$, and $\boldsymbol{\beta}$ are trainable parameters, μ_* are some hyperparameters, and σ is the sigmoid function. The symbol $\odot$ denotes an elementwise product, and exponentiation to a vector is intended elementwise. The states in the recursion are initialized as $\mathbf{a} = \mathbf{b} = \mathbf{0}$.

With some abuse of notation, also denoting with $\mathbf{u}_{y-1} \in \mathbb{R}^F$ the input to the channel mixing module and its corresponding output as $\mathbf{o}_{y-1} \in \mathbb{R}^F$, the module performs the following operations:

$$\mathbf{r}_{y-1} = \mathbf{W}'_r\left(\mu'_r \mathbf{u}_{y-1} + \left(1-\mu'_r\right)\mathbf{u}_{y-2}\right) \tag{8}$$
$$\mathbf{k}_{y-1} = \mathbf{W}'_k\left(\mu'_k \mathbf{u}_{y-1} + \left(1-\mu'_k\right)\mathbf{u}_{y-2}\right) \tag{9}$$
$$\mathbf{o}_{y-1} = \sigma\left(\mathbf{r}_{y-1}\right) \odot \left(\mathbf{W}'_v \max\left(\mathbf{k}_{y-1}, 0\right)^2\right) \tag{10}$$

where $\mathbf{W}'_*$ are trainable parameters, μ'_* are some hyperparameters, and σ is the sigmoid function.

It can be noticed that the recursion requires a limited amount of memory to be implemented. In particular, besides the features of the current line, the only feature vectors that need to be stored are the states $\mathbf{a}_{y-2}$ and $\mathbf{b}_{y-2}$ and features of the previous line $\mathbf{u}_{y-2}$. This needs to be done for each layer of line and channel mixing that is used. However, this operation allows keeping a memory of past lines and using the attention mechanism for the prediction of the next line and is significantly advantageous in terms of memory consumption compared to keeping the features of a large number of lines. We refer the reader to [23] for details about a numerically-stable implementation.

The output $\mathbf{o}_{y-1}$ of the last channel mixing module in the line predictor is summed with the input of the module in a

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

residual fashion, as depicted in Fig. 3, to generate $\hat{\mathbf{u}}_y \in \mathbb{R}^F$, i.e., a feature-space prediction of the colocated pixel in line y, as shown in Algorithm 1.

D. Spectral Predictor

The output $\hat{\mathbf{u}}_y$ of the line predictor is a feature vector for each spatial–spectral pixel that can be thought to be a prediction of the features of line y. While this captures the spatial correlation in the across- and along-track directions, the model has not yet exploited correlation in the spectral dimension. We propose to first compute a feature-domain spatial prediction residual and then model its spectral correlation. In particular, the pixels in line y are encoded with the encoder in Section III-B, and the difference between them and the output of the line predictor applied to line $y-1$ forms the spectral sequence of feature-domain spatial residuals

$$\mathbf{\Delta}_{y,x,z} = \mathbf{u}_{y,x,z} - \hat{\mathbf{u}}_{y,x,z} \tag{11}$$

which, in the following, denotes in short as $\mathbf{\Delta}_z$, omitting subscripts x, y. Notice that $\mathbf{\Delta}_{y,x,z}$ requires pixel $I^q_{y,x,z}$ for its computation. Therefore, this needs to have been already decoded, and its availability is ensured by treating the first band as a special case, as discussed in Section III-F.

A causal model over the $\mathbf{\Delta}_z$ sequence is needed to obtain features describing the pixel to be predicted. For this spectral prediction model, we propose to also use RWKV blocks. However, for a lossless (or prequantized) compressor, the parallel implementation typically reserved for training is used rather than the recurrent implementation described in Section III-C in order to ensure high throughput. This consists of a sequence of N_{sp} band mixing and channel mixing blocks, with the same architecture previously depicted in Fig. 3. The parallel implementation keeps the features for all the bands in memory and computes[1]

$$\mathbf{r}_{z-1} = \mathbf{W}_r(\mu_r \mathbf{\Delta}_{z-1} + (1-\mu_r)\mathbf{\Delta}_{z-2}) \tag{12}$$

$$\mathbf{k}_{z-1} = \mathbf{W}_k(\mu_k \mathbf{\Delta}_{z-1} + (1-\mu_k)\mathbf{\Delta}_{z-2}) \tag{13}$$

$$\mathbf{v}_{z-1} = \mathbf{W}_v(\mu_v \mathbf{\Delta}_{z-1} + (1-\mu_v)\mathbf{\Delta}_{z-2}) \tag{14}$$

$$\mathbf{p}_{z-1} = \frac{\sum_{i=1}^{z-2} e^{-(z-1-i)\boldsymbol{\alpha}+\mathbf{k}_i} \odot \mathbf{v}_i + e^{\boldsymbol{\beta}+\mathbf{k}_{z-1}} \odot \mathbf{v}_{z-1}}{\sum_{i=1}^{z-2} e^{-(z-1-i)\boldsymbol{\alpha}+\mathbf{k}_i} + e^{\boldsymbol{\beta}+\mathbf{k}_{z-1}}} \tag{15}$$

$$\mathbf{o}_{z-1} = \mathbf{W}_o(\sigma(\mathbf{r}_{z-1}) \odot \mathbf{p}_{z-1}) \tag{16}$$

for the band mixing block and

$$\mathbf{r}_{z-1} = \mathbf{W}'_r\left(\mu'_r \mathbf{u}_{z-1} + \left(1-\mu'_r\right)\mathbf{u}_{z-2}\right) \tag{17}$$

$$\mathbf{k}_{z-1} = \mathbf{W}'_k\left(\mu'_k \mathbf{u}_{z-1} + \left(1-\mu'_k\right)\mathbf{u}_{z-2}\right) \tag{18}$$

$$\mathbf{o}_{z-1} = \sigma(\mathbf{r}_{z-1}) \odot \left(\mathbf{W}'_v \max(\mathbf{k}_{z-1}, 0)^2\right) \tag{19}$$

for the channel mixing block. The output $\mathbf{o}_{z-1}$ of the last channel mixing module in the spectral predictor is summed with the input of the module in a residual fashion, as depicted in Fig. 3, to generate a feature vector $\hat{\mathbf{\Delta}}_z$ for each pixel, which should ideally represent the spatial–spectral residual in a feature domain. Also, notice that spectral prediction is performed on the difference sequence $\mathbf{\Delta}_z$ rather than on the outputs of the line predictors as this allows to use the information of the pixel in the same line to be encoded but from previous bands, which is typically the highest source of correlation.

[1]With some abuse of notation, we reuse symbols from the line predictor, but they represent different activations and weights.

E. Decoder

A decoder neural network produces a prediction of the raw pixel value from the feature-domain representation of the spatial prediction $\hat{\mathbf{u}}_{y,x,z}$ and spatial–spectral error $\hat{\mathbf{\Delta}}_{y,x,z}$, as shown in Algorithm 1. This is obtained by feeding their sum to a sequence of N_{dec} blocks composed of 1×1 convolution, LayerNorm, and nonlinear activation.

The predicted pixel value is then denormalized, rounded to the nearest integer and the difference with respect to the original pixel value forms the prediction error, which is entropy-coded with a suitable technique. Notice that this approach differs somewhat from works addressing lossless compression of natural images with deep learning [13]. In those works, it is more typical to let the neural network produce the prediction as a probability distribution over the set of possible symbols (typically 256 for 8-bit images). However, the large number of symbols (e.g., 2^{16}) of satellite images poses efficiency challenges in terms of computation and memory requirements and incurs the "softmax bottleneck" issue due to the number of classes exceeding the number of features [39]. The proposed floating-point regression with rounding solves these issues but incurs subtle numerical conditions that must be managed to ensure decodability. Whenever there is a mismatch in the hardware-software stack between the compressor and the decompressor, the prediction will only be accurate down to numerical precision, i.e., about seven significant digits on the normalized value, for computations in FP32. This means that a numerical perturbation might cause the predicted value to cross the rounding threshold, causing a decoding error. In order to avoid this, whenever the prediction is closer to the rounding threshold than 10^{-3} (in unnormalized integer digital numbers), we signal the side of the threshold as extra information. This choice has been experimentally verified to ensure correct decoding on the entire HySpecNet-11k test set. The side information is entropy-coded and, in our tests, incurred a small penalty of 0.02 bpppc, which is already included in all experimental results.

F. Special Cases

In order to ensure causality of the model, the first line for all bands and the entire first band need to be compressed with a separate method. For the first line, we use a simple differential pulse coded modulation (DPCM) encoder where the prediction error is computed as

$$E_{0,x,z} = \begin{cases} I^q_{0,0,0}, & \text{for } x=0, z=0 \\ I^q_{0,x,z} - I^q_{0,x-1,0}, & \text{for } x>0,\ z=0 \\ I^q_{0,x,z} - I^q_{0,x,z-1} & \text{for } x\geq 0,\ z>0. \end{cases}$$

For the first band, we employ spatial prediction only by feeding the output of the line predictor to a dedicated decoder neural network with architecture comparable to that described in Section III-E; the prediction error is then entropy-coded.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LINERWKV ARCHITECTURE CONFIGURATIONS

Model Size	Params	N_{enc}	N_{lp}	N_{sp}	N_{dec}	F	FLOPS/sample
XS	30k	1	2	2	1	32	120k
S	135k	2	2	2	2	64	508k
M	286k	4	4	4	4	64	1M
L	900k	4	6	6	4	96	3.2M

G. Training Versus Inference

The description in Sections III-B–III-F follows the inference behavior. A few important differences are present for training in order to ensure scalability when using large datasets. First, the recurrent implementation of the line predictor is replaced with the parallel implementation already described in Section III-D. This improves efficiency as it enables processing several lines at the same time, and in fact, it is one of the main advantages over traditional RNNs. Moreover, we remark an interesting property of the overall design, which is useful for efficient training: all operations performed after the line encoder are columnwise independent. In fact, the predictors work on lines and bands but do not mix columns, and the use of 1×1 convolution in the decoder also keeps this property. This is useful to reduce memory consumption during training by subsampling random subsets of columns after encoding, if desired.

LineRWKV is trained by minimizing the ℓ_1 loss between the predicted and true pixel values. Notice that this means that we only optimize for lossless compression while never accounting for quantization during training. Nevertheless, lossy rate distortion tests still show excellent performance, as reported in Section IV.

IV. EXPERIMENTAL RESULTS

In this section, we analyze the performance of LineRWKV in terms of compression efficiency with respect to state-of-the-art approaches to hyperspectral image compression. Moreover, we show how LineRWKV behaves when a different satellite is targeted compared to its original training. Finally, we report throughput and memory usage on a 7-W low-power device as a proof of concept for potential onboard implementation on embedded devices. Code and pretrained models are available at https://github.com/diegovalsesia/linerwkv.

A. Experimental Setting

Our main experimental results are based on training and testing LineRWKV on the recently introduced HySpecNet-11k dataset [40]. This is the largest curated dataset of hyperspectral images currently available, composed of 11 483 nonoverlapping patches of size $128 \times 128 \times 224$ acquired by the EnMAP satellite with a ground sampling distance of 30 m. The authors provide standard train–test splits for benchmarking compression algorithms. In particular, we use the "hard" split where patches in the test set belong to entirely separate tiles with respect to the training patches. Preprocessing discards some bands, resulting in a total of 202, and clips values between 0 and 10 000. We also test the method on two additional datasets: a collection of images from the PRISMA satellite that we assembled and the dataset released for the DFC 2018 challenge [41]. For the former dataset, we collected 110 images of size $1000 \times 1000 \times 239$ from the PRISMA satellite [42], partitioned into 100 for training and ten for testing. These images have been collected from all over the world to have a highly varied set of scenes, and the test locations are strictly disjoint from the train locations. For the experiments, we used the 66 visible near-infrared (VNIR) bands. For the latter dataset, an aerial scene of Houston of size $4172 \times 1202 \times 50$ has been partitioned into $128 \times 128 \times 50$ subimages. HyspecNet-11k data are preprocessed with clipping to 10 000 maximum intensity, thus having a dynamic range of roughly 14 bits, while the PRISMA and DFC2018 images are over 16 bits.

We consider several baseline methods for hyperspectral image compression, focusing on low-complexity methods. In particular, we choose the state-of-the-art CCSDS standard for onboard lossless and near-lossless hyperspectral predictive compression, i.e., CCSDS-123.0-B-2 [22] and RWA [29] for an alternative state-of-the-art approach to lossless compression based on transforms. As for state-of-the-art deep learning approaches, we evaluate the 1-D-CAE [14], spectral signals compressor network (SSCNet) [15] methods, and the recent method by Verdú et al. [21]; we remark that these methods only perform lossy compression. All methods have been retrained, using the reference implementation by the HySpecNet-11k authors for the former two and the authors' code for the latter. The 1-D-CAE [14] is a convolutional autoencoder, which limits computational complexity by working only in the spectral dimension, as it is known that most of the redundancy lies in interband correlation. SSCNet [15] also follows a spectral autoencoding approach. The work by Verdú et al. [21] manages spatial–spectral autoencoding at low complexity by introducing a new design for a variational autoencoder with a hyperprior network, operating on groups of bands and with ad hoc normalization schemes.

We use the sample-adaptive Golomb encoder defined by the CCSDS-123.0-B-2 standard [22] as an entropy coder for the positive-mapped prediction residuals, both for our method and the CCSDS method to ensure a fair comparison; it is obvious that the performance could be improved employing an arithmetic coder, especially at rates below 2 bpppc, and as a matter of fact, the Golomb coder is not capable of producing rates below 1 bpppc. The other methods use the same entropy encoder as proposed in their respective works. The CCSDS predictor is used in its full mode with wide neighbor-oriented local sums, and three prediction bands, a configuration that typically provides the best tradeoff of compression ratio and complexity.

Several configurations for LineRWKV have been tested in order to validate its scaling potential as a function of the number of encoders, decoders, predictor layers, and number of features. Table I reports the details for four configurations of choice, ranging from extra small (XS) to large (L). Training LineRWKV has been done over four Nvidia A100 40-GB GPUs. In order to limit training memory requirements and speed up convergence, a random subset of 16 contiguous bands has been used in the initial training phases, followed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

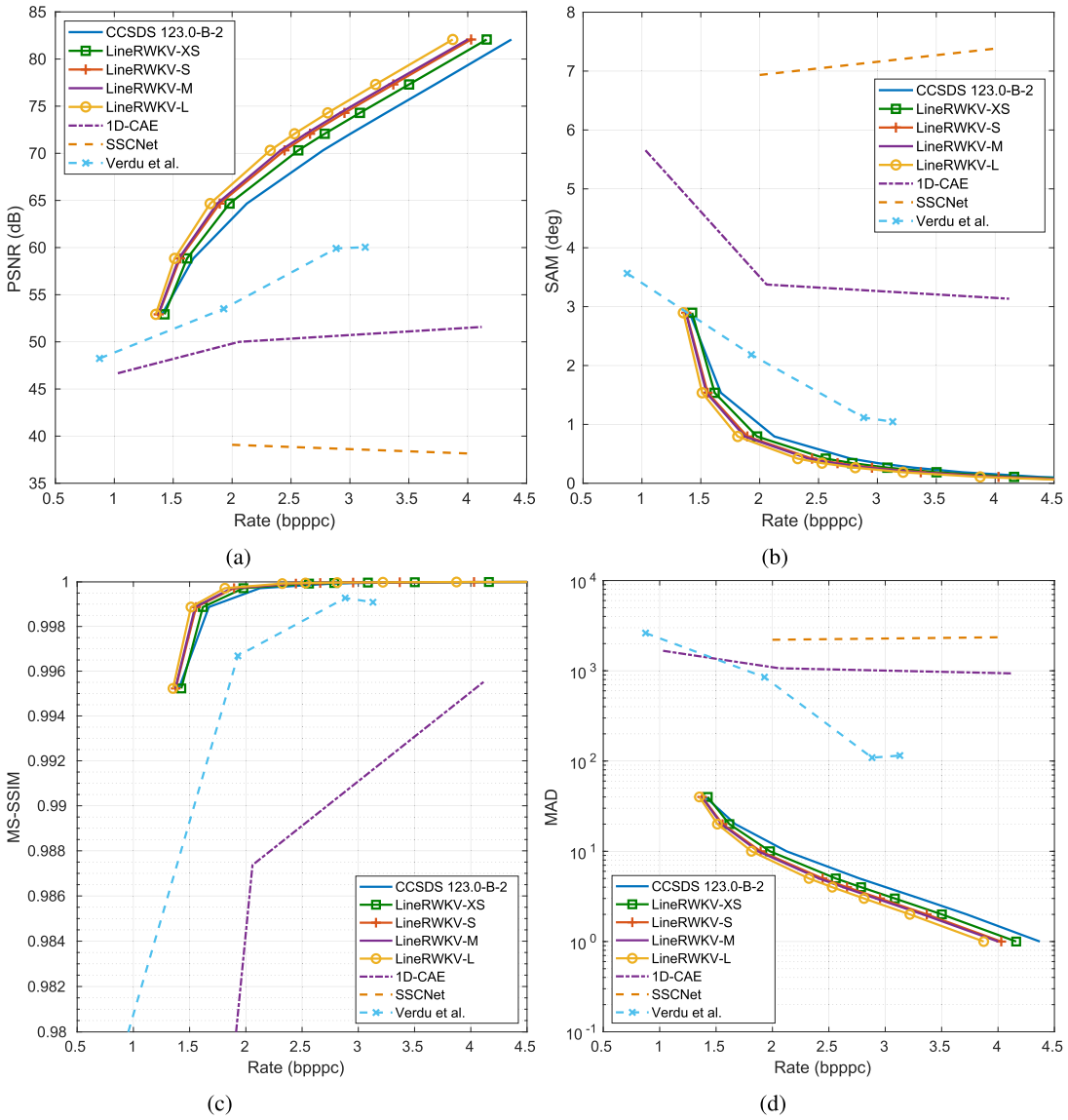

Fig. 4. Rate-distortion performance comparison on the HySpecNet-11k hard test set. SSCNet multiscale structural similarity index measure (MS-SSIM) is outside the figure scale at 0.914 at 2 bpppc and 0.891 at 4 bpppc. (a) PSNR. (b) SAM. (c) MS-SSIM. (d) MAD.

TABLE II
LOSSLESS RATE (BPPPC) ON HYSPECNET-11K HARD TEST SET

Model size	LineRWKV	CCSDS [22]	RWA [29]	Diff. CCSDS	Diff. RWA
XS	**5.647**	5.801	5.772	-0.154	-0.125
S	**5.521**	5.801	5.772	-0.280	-0.251
M	**5.510**	5.801	5.772	-0.291	-0.262
L	**5.370**	5.801	5.772	-0.431	-0.402

by fine-tuning with all the available bands and a smaller batch size. For the M configuration, the initial batch size was 8 without any column sampling; in the fine-tuning phase with all bands, the batch size is 4 with a subsampling of 16 columns. Learning rate was linearly decreased from 10^{-4} to 10^{-6} over 4000 epochs. We remark that memory requirements during training are large due to the desire to process as much of the hyperspectral cube as possible concurrently with a large enough batch size. This is not reflected in the inference phase, which is memory-efficient, as shown in Section IV-C.

B. Results on HySpecNet-11k and Transfer Learning to Other Sensors

Our main experiment assesses the rate-distortion performance of all methods when tested on the hard test split of HySpecNet-11k, as well as on the PRISMA and DFC2018 datasets.

We first start with lossless compression. Table II reports a comparison between the rates obtained by LineRWKV, CCSDS-123.0-B-2 [22], and RWA [29] on HySpecNet-11k. RWA is sometimes slightly better than CCSDS 123.0-B-2 at lossless compression, but both are significantly outperformed by the proposed LineRWKV.

We are also interested in studying how well a LineRWKV model trained on a dataset from a specific satellite behaves when it is applied to a different satellite. This is important since it is likely that a large-scale dataset of images representative of a new satellite is not available before launch. However, it is also conceivable that a certain number of images can be losslessly transmitted with a suboptimal compressor

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

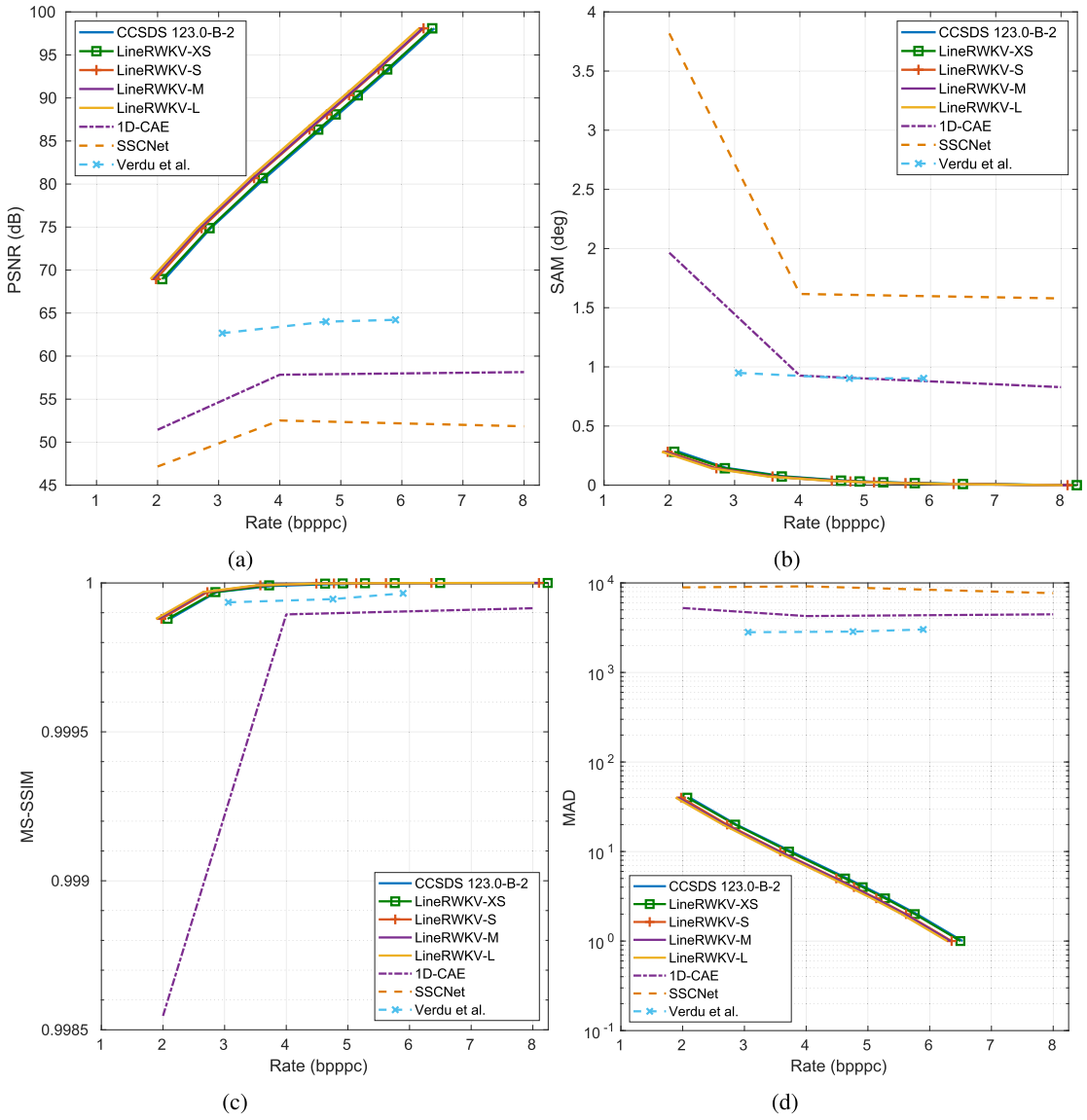

Fig. 5. Rate-distortion performance comparison on the PRISMA test set. SSCNet MS-SSIM is outside the figure scale at 0.988 at 4 bpppc and 0.989 at 8 bpppc. (a) PSNR. (b) SAM. (c) MS-SSIM. (d) MAD.

TABLE III
TRANSFER LEARNING—LOSSLESS RATE

Dataset	Model size	LineRWKV (zero-shot)	LineRWKV (finetune)	CCSDS [22]	RWA [29]
PRISMA	XS	8.431	8.214	8.135	**8.105**
	S	8.312	**8.076**	8.135	8.105
	M	8.188	**7.974**	8.135	8.105
	L	8.093	**7.919**	8.135	8.105
DFC2018	XS	7.484	**7.315**	7.364	7.403
	S	7.378	**7.216**	7.364	7.403
	M	7.353	**7.175**	7.364	7.403
	L	7.334	**7.107**	7.364	7.403

in the preoperational phase in order to fine-tune and update the compressor model itself. For this experiment, we use the PRISMA and DFC2018 datasets. Table III reports the lossless rate compared to CCSDS-123.0-B-2 and RWA on the PRISMA and DFC2018 datasets in two settings of interest: *zero-shot transfer*, i.e., the model trained on HySpecNet-11k is used as is on DFC2018 and PRISMA, and after *fine-tuning*. It can be noticed that the domain gap between EnMAP and the PRISMA and DFC2018 sensors affects compression performance in the zero-shot case, which is now either slightly outperformed by or close to the CCSDS algorithm. However, fine-tuning with a modest number of images allows for recovering superior performance.

For lossy compression, we are mostly interested in the high-quality and high-rate regimes (typically $\geq$ 2 bpppc) as these are the most relevant for space missions. We consider the following metrics for quality evaluation: peak signal-to-noise ratio (PSNR), MS-SSIM [43], spectral angle mapper (SAM) score [44], and maximum absolute distortion (MAD). Figs. 4–6 show the average values of the evaluation metrics obtained over the entire test set as a function of the rate for all the methods under study on the HySpecNet-11k, PRISMA, and DFC2018 datasets, respectively. For PRISMA and DFC2018, we are reporting the results after fine-tuning. It can be noticed that existing deep learning approaches are significantly outperformed by the CCSDS standard at high rates. This is due

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

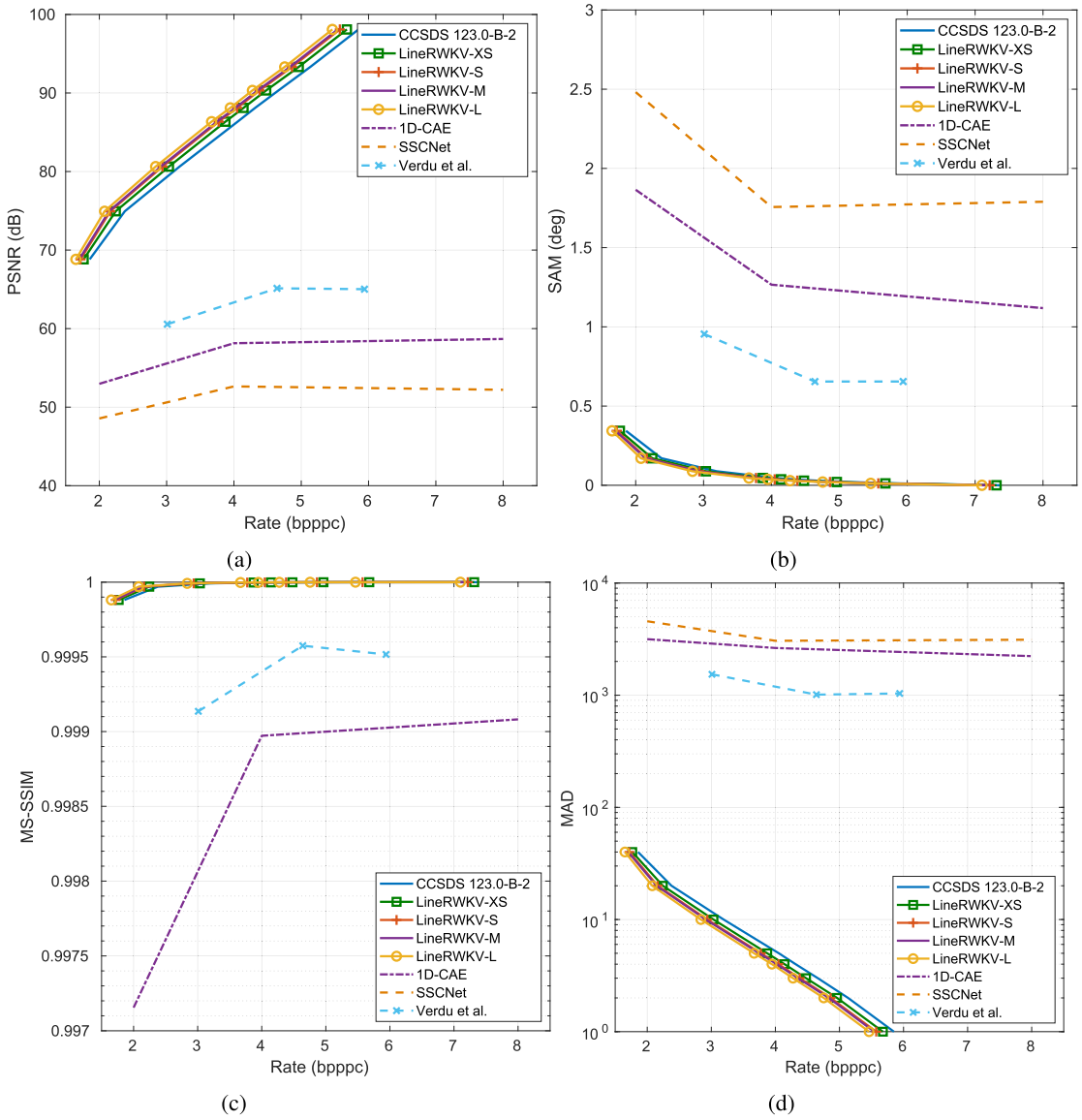

Fig. 6. Rate-distortion performance comparison on the DFC2018 test set. SSCNet MS-SSIM is outside the figure scale at 0.995 at 4 bpppc and 0.994 at 8 bpppc. (a) PSNR. (b) SAM. (c) MS-SSIM. (d) MAD.

to the current focus of the literature on hyperspectral image compression toward very low rates, which finds its motivation in complex challenges toward scaling autoencoder neural network designs to also work well at high rates while maintaining acceptable complexity. As a sanity check, we remark that the reported PSNR values on the hard split are consistent with published results on the easy split by the HySpecNet-11k authors.

The results on the MAD metric also raise an interesting point of discussion since the state-of-the-art deep learning approaches are two orders of magnitude worse than LineRWKV and CCSDS-123.0-B-2. This is expected since the traditional autoencoder design does not constrain the maximum error in any way, possibly resulting in localized very high errors. On the other hand, predictive compressors naturally limit maximum distortion to half of the quantization step size, both for the prequantized approach followed by LineRWKV and the in-loop quantization of CCSDS-123.0-B-2. Fig. 7 shows a visualization of the maximum absolute error and reports histograms of the full absolute error distribution, at a rate point close to 4 bpppc for all methods. LineRWKV and CCSDS-123.0-B-2 set the quantization step size to $Q = 21$, so the maximum error in constrained to a value of 10, and we can see that it shows a roughly uniform distribution. On the other hand, the deep autoencoding approaches exhibit an exponential error distribution with tails with significantly larger errors. A guarantee of the maximum error is critical to ensure image reliability in real missions, thus showing an important advantage of the proposed LineRWKV method.

Regarding the overall comparison with CCSDS-123.0-B-2, we can notice that it is closer in performance than deep learning approaches, but LineRWKV always outperforms it, even in the XS configuration. This is also shown in Table IV, which reports the Bjontegaard Delta metric (BD-Rate) with respect to PSNR. We remark that since both LineRWKV and CCSDS-123.0-B-2 are predictive schemes working at high

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

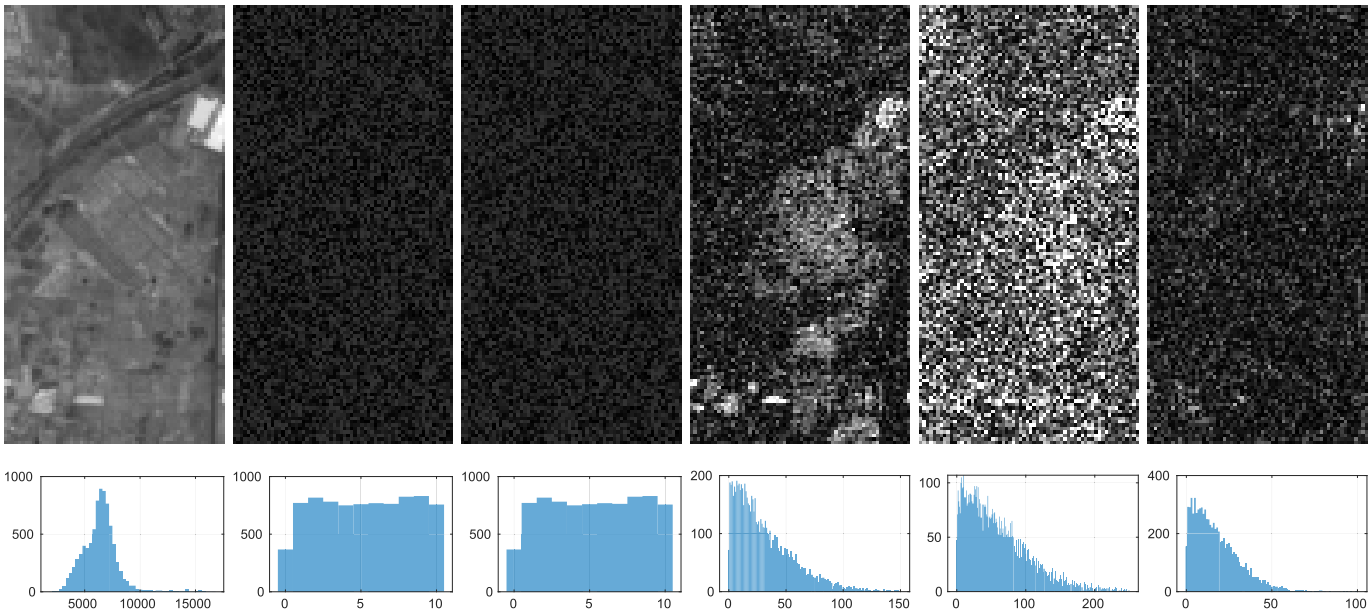

Fig. 7. Absolute error at 4 bpppc. (Left to right) Ground-truth image crop from the PRISMA test set, LineRWKV-XS, CCSDS-123.0-B-2, 1-D-CAE, SSCNet, and Verdú et al. Note that the white level is 50 for LineRWKV and CCSDS and 150 for 1-D-CAE, SSCNet, and Verdú et al. to improve visibility.

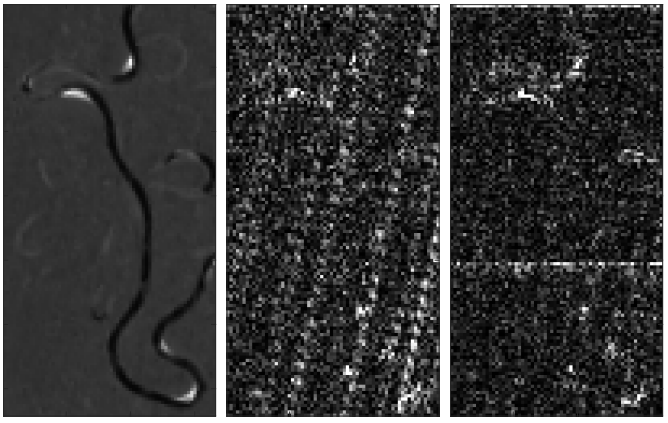

Fig. 8. (Left to right) Band 187 of a test image (colormap $[0, 4000]$) and positive-mapped prediction residuals (colormap $[0, 100]$) for CCSDS-123.0-B-2 and LineRWKV-XS.

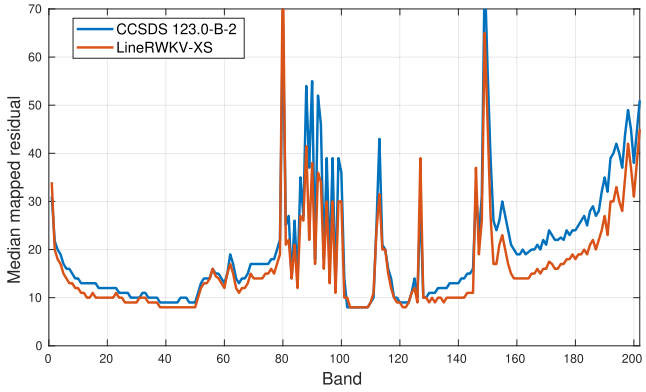

Fig. 9. Positive-mapped prediction residual for CCSDS-123.0-B-2 and LineRWKV-XS. Median, over the entire HySpecNet-11k hard test set, of spatial medians.

rates, their quality metrics are essentially the same, differing only in rate.

Fig. 8 shows a qualitative comparison of the positive-mapped prediction residuals of LineRWKV-XS and CCSDS-123.0-B-2 for a band in a test image. It can be noticed that some complex correlation patterns are discovered by the neural network and removed, leading to smaller residuals. On the other hand, Fig. 9 reports the median positive-mapped prediction residual as a function of band index for the entire HySpecNet-11k hard test set. We can see that LineRWKV-XS provides smaller residuals across basically all the 202 bands, with the largest gains observed for bands 130–202.

We also report an example of the performance of a downstream application, where compressed images are used to address land cover classification. The Pavia dataset is used for this task with models trained on HyspecNet-11k. Table V reports the accuracy as a function of rate when images compressed by LineRWKV and 1-D-CAE are used for training and testing a simple model based on support vector machines (SVMs). Although this particular application is fairly robust to compression, we can see that LineRWKV provides a smaller degradation of accuracy as the bit rate decreases. In general, the performance of downstream applications might be more or less sensitive to loss of information due to compression and should be carefully assessed on a task-by-task basis.

TABLE IV

BD-RATE OF LINERWKV WITH RESPECT TO CCSDS-123.0-B-2 FOR LOSSY COMPRESSION

Model size	HyspecNet-11k	PRISMA	DFC2018
XS	-5.27%	-0.89%	-4.53%
S	-9.05%	-4.36%	-7.13%
M	-9.94%	-5.11%	-7.84%
L	-12.67%	-6.37%	-8.45%

It is also worth noticing that there is a clear performance scaling, with increasing rate-distortion gains both in lossless and lossy compression, in the LineRWKV model, as model complexity is increased from the XS to L configuration. For example, on HySpecNet-11k, the XS configuration reports an

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

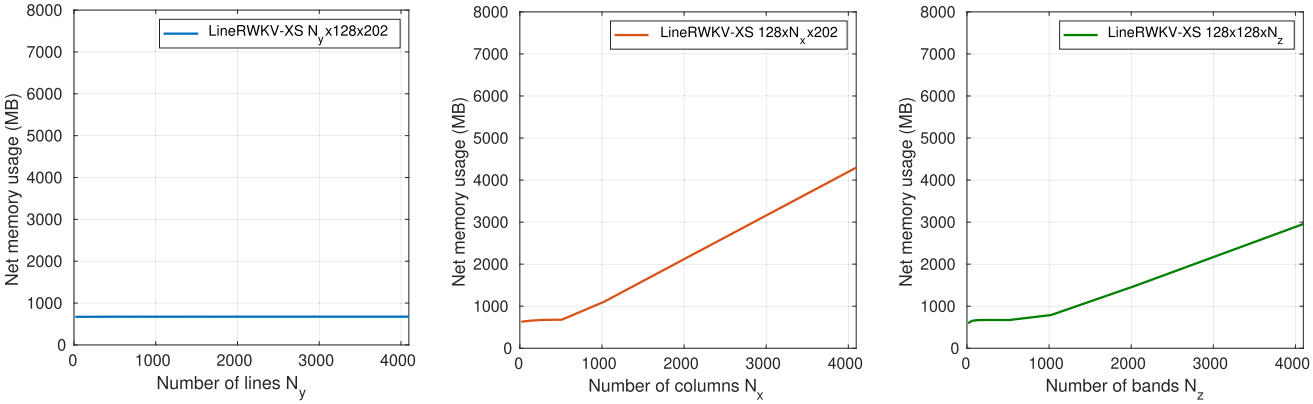

Fig. 10. Scaling of LineRWKV-XS memory usage on the Nvidia Jetson Orin Nano 8 GB as a function of number of lines, columns, and bands.

TABLE V

LAND COVER CLASSIFICATION FROM COMPRESSED IMAGES. SVM ACCURACY ON THE PAVIA DATASET

Model	Rate			
	Orig.	8 bpppc	4 bpppc	2 bpppc
LineRWKV-XS	**91.15%**	**91.15%**	**91.14%**	**90.94%**
1D-CAE	**91.15%**	88.92%	87.85%	85.33%

TABLE VI

ABLATION OF THE DESIGN OF SPECTRAL PREDICTOR

Model size	RWKV		Causal conv.	
	Rate	Throughput	Rate	Throughput
XS	5.647	511,345	5.764	595,962
M	5.510	173,078	5.593	334,476

already substantial 0.154-bpppc gain over lossless CCSDS-123.0-B-2, while the L configuration achieves a massive 0.431-bpppc gain. Although the L configuration is too complex for practical onboard usage, it serves as a validation that LineRWKV has potential scaling and is not significantly bottlenecked in its design.

As a term of comparison regarding complexity, the work by Verdú et al. [21] reports a complexity of about 17k FLOPs/sample compared to the 120k FLOPs/sample of LineRWKV-XS. However, we also tested a variant of the method where we increased the number of hidden and latent features to approximately match 120k FLOPs/sample. This is the point with a rate equal to 3.130 bpppc reported in Fig. 4, which, however, does not seem to provide significant improvements, highlighting that the design has some bottleneck preventing it from reaching low distortion values at higher rates.

C. Performance on Low-Power Hardware

In order to validate the low-complexity inference of LineRWKV, we test its performance on a low-power embedded system, namely, the Nvidia Jetson Orin Nano. The platform has 8 GB of shared CPU-GPU memory and a maximum power of 7 W. We test the XS configuration in single-precision floating point (FP32). For the HySpecNet-11k images ($128 \times 128 \times 202$), the peak memory usage due to LineRWKV is 673 MB, which is a quite modest value in the realm of deep learning-based processing of hyperspectral images. The measured latency is 6.472 s, resulting in a throughput of 511 345 samples/s. Fig. 10 shows how memory scales as a function of number of lines, columns, and bands. First, we can notice that the recurrent property over the along-track direction allows constant memory usage regardless of the number of lines. For what concerns columns and bands, we can see that scaling is fairly linear, in contrast with the quadratic dependence in traditional Transformers, with a floor for small image size. Throughput is roughly constant at around 500k samples/s for all image sizes except for the very small ones, which also confirms the linear scaling of complexity.

It must be remarked that the current implementation is far from optimized. Indeed, several optimizations are possible, which would substantially raise the throughput and might be worth investigating in future works. Few examples include model compilation for the target accelerator (e.g., via TensorRT) and mixed-precision inference where parts of the model run integer-quantized or in floating-point half precision. Our proposed design also does not exploit any spatial downsampling: if the line and spectral predictors could be run on a downsampled cube, throughput would be increased by a factor close to the downsampling factor. However, this necessitates careful decoder design not to reduce rate-distortion performance.

D. Spectral Predictor Ablation

In this section, we evaluate the choice of RWKV blocks as a spectral prediction mechanism against an alternative approach, as presented in [25]. The alternative approach adopts causal convolution operations such that the receptive field expands only toward past bands. In particular, blocks are repeated to form the spectral predictor; each block is composed of 1-D causal convolution with a kernel length of three bands, layer normalization, Gaussian error linear unit (GeLU) nonlinearity, and an attention operation. This operation computes an attention mask from the output of the GeLU nonlinearity with two size-1 convolutions interleaved by GeLU and a final sigmoid activation. The resulting mask is multiplied elementwise to the input of the attention operation, and the result is summed to it. Comparisons are made to have a similar number of trainable parameters with respect to the design with RWKV blocks.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

Table VI reports a comparison between the lossless rate and throughput on the Nvidia Jetson Orin Nano of some configurations of the proposed LineRWKV model with RWKV-based spectral predictor and those of the model with causal convolution-based spectral predictor. We can see that the causal convolution method is faster but not as effective in terms of compression ratio.

V. Conclusion and Limitations

We presented a novel design of a low-complexity neural network for the compression of hyperspectral images with the goal of using onboard satellites. In particular, we showed that the memory and complexity bottleneck of current autoencoder designs can be overcome by adopting a predictive coding approach with a neural network working line by line based on hybrid recurrent-attentive operations. This leads to a substantial improvement in rate-distortion performance, resulting in the first deep-learning approach capable of outperforming CCSDS-123.0-B-2 and having modest memory requirements.

While LineRWKV represents a substantial improvement in the performance of low-complexity deep learning compressors, some limitations need to be addressed in future work. In particular, complexity needs to be further reduced to achieve throughput comparable with current in-flight implementations. We conjecture that a possible avenue could be reducing the columns and bands into a more compact latent space to be used for prediction, essentially creating a hybrid method between autoencoding and prediction. The numerical stability of the prediction, while managed with a small overhead in the current scheme, also deserves further investigation. Moreover, optimal training may require a substantial amount of GPU memory for sensors with hundreds of bands. The solution to this does not seem trivial, as we have shown that lighter methods for spectral prediction, such as causal convolution, do not seem as effective as RWKV. Finally, our scaling experiments show that increasing the number of parameters of LineRWKV brings improvements in rate-distortion performance for the same architectural design. However, at the moment, it is unclear whether this is an intrinsic limitation of neural networks, requiring larger scales to be more effective, or if the design could be made more parameter efficient.

Acknowledgment

This study was carried out within the FAIR - Future Artificial Intelligence Research and received funding from the European Union Next-GenerationEU (PIANO NAZIONALE DI RIPRESA E RESILIENZA (PNRR) – MISSIONE 4 COMPONENTE 2, INVESTIMENTO 1.3 – D.D. 1555 11/10/2022, PE00000013). This manuscript reflects only the authors' views and opinions, neither the European Union nor the European Commission can be considered responsible for them.

References

[1] M. Jiang et al., "GraphGST: Graph generative structure-aware transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5504016.

[2] S. K. Roy, A. Deria, C. Shah, J. M. Haut, Q. Du, and A. Plaza, "Spectral–spatial morphological attention transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5503615.

[3] S. Mei, C. Song, M. Ma, and F. Xu, "Hyperspectral image classification using group-aware hierarchical transformer," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5539014.

[4] Y. Su et al., "DAAN: A deep autoencoder-based augmented network for blind multilinear hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5512715.

[5] M. E. Paoletti, S. Moreno-Álvarez, Y. Xue, J. M. Haut, and A. Plaza, "AAtt-CNN: Automatic attention-based convolutional neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5511118.

[6] Z. Wu and B. Wang, "Transformer-based autoencoder framework for nonlinear hyperspectral anomaly detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5508015.

[7] Y. Dua, V. Kumar, and R. S. Singh, "Comprehensive review of hyperspectral image compression algorithms," *Opt. Eng.*, vol. 59, no. 9, Sep. 2020, Art. no. 090902.

[8] J. Ballé, V. Laparra, and E. P. Simoncelli, "End-to-end optimized image compression," 2016, *arXiv:1611.01704*.

[9] L. Zhou, C. Cai, Y. Gao, S. Su, and J. Wu, "Variational autoencoder for low bit-rate image compression," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2018, pp. 2617–2620.

[10] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, "Variational image compression with a scale hyperprior," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–23.

[11] J. Ballé et al., "Nonlinear transform coding," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 2, pp. 339–353, Feb. 2021.

[12] W. Jiang, J. Yang, Y. Zhai, P. Ning, F. Gao, and R. Wang, "MLIC: Multi-reference entropy model for learned image compression," in *Proc. 31st ACM Int. Conf. Multimedia*, Oct. 2023, pp. 7618–7627.

[13] F. Mentzer, E. Agustsson, M. Tschannen, R. Timofte, and L. van Gool, "Practical full resolution learned lossless image compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10629–10638.

[14] J. Kuester, W. Gross, and W. Middelmann, "1D-convolutional autoencoder based hyperspectral data compression," *Int. Arch. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 43, pp. 15–21, Jun. 2021.

[15] R. La Grassa, C. Re, G. Cremonese, and I. Gallo, "Hyperspectral data compression using fully convolutional autoencoder," *Remote Sens.*, vol. 14, no. 10, p. 2472, May 2022.

[16] V. A. de Oliveira et al., "Reduced-complexity end-to-end variational autoencoder for on board satellite image compression," *Remote Sens.*, vol. 13, no. 3, p. 447, Jan. 2021.

[17] Y. Dua, R. S. Singh, K. Parwani, S. Lunagariya, and V. Kumar, "Convolution neural network based lossy compression of hyperspectral images," *Signal Process., Image Commun.*, vol. 95, Jul. 2021, Art. no. 116255.

[18] Y. Guo, Y. Tao, Y. Chong, S. Pan, and M. Liu, "Edge-guided hyperspectral image compression with interactive dual attention," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2022, Art. no. 5500817.

[19] Y. Guo, Y. Chong, and S. Pan, "Hyperspectral image compression via cross-channel contrastive learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5513918.

[20] J. Kuester, W. Gross, S. Schreiner, W. Middelmann, and M. Heizmann, "Adaptive two-stage multisensor convolutional autoencoder model for lossy compression of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5530022.

[21] S. M. I. Verdú, J. Ballé, V. Laparra, J. Bartrina-Rapesta, M. Hernández-Cabronero, and J. Serra-Sagristà, "A scalable reduced-complexity compression of hyperspectral remote sensing images using deep learning," *Remote Sens.*, vol. 15, no. 18, p. 4422, Sep. 2023.

[22] Consultative Committee for Space Data Systems (CCSDS). (Feb. 2019). *Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression*. Blue Book. [Online]. Available: https://public.ccsds.org/Pubs/123x0b2c3.pdf

[23] B. Peng et al., "RWKV: Reinventing RNNs for the transformer era," 2023, *arXiv:2305.13048*.

[24] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.

[25] D. Valsesia, T. Bianchi, and E. Magli, "Hybrid recurrent-attentive neural network for onboard predictive hyperspectral image compression," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2024, pp. 7898–7902.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.

[26] L. Santos, L. Berrojo, J. Moreno, J. Fco. López, and R. Sarmiento, "Multispectral and hyperspectral lossless compressor for space applications (HyLoC): A low-complexity FPGA implementation of the CCSDS 123 standard," *IEEE J. Sel. Topics Appl. Earth Obser. Remote Sens.*, vol. 9, no. 2, pp. 757–770, Feb. 2016.

[27] Consultative Committee for Space Data Systems (CCSDS). (Sep. 2017). *Image Data Compression*. Blue Book. [Online]. Available: https://public.ccsds.org/Pubs/122x0b2.pdf

[28] Consultative Committee for Space Data Systems (CCSDS). (Sep. 2017). *Spectral Preprocessing Transform for Multispectral and Hyperspectral Image Compression*. Blue Book. [Online]. Available: https://public.ccsds.org/Pubs/122x1b1.pdf

[29] N. Amrani, J. Serra-Sagristà, V. Laparra, M. W. Marcellin, and J. Malo, "Regression wavelet analysis for lossless coding of remote-sensing data," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 9, pp. 5616–5627, Sep. 2016.

[30] S. Álvarez-Cortés, J. Serra-Sagristà, J. Bartrina-Rapesta, and M. W. Marcellin, "Regression wavelet analysis for near-lossless remote sensing data compression," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 2, pp. 790–798, Feb. 2020.

[31] Y. Chong, L. Chen, and S. Pan, "End-to-end joint spectral–spatial compression and reconstruction of hyperspectral images using a 3D convolutional autoencoder," *J. Electron. Imag.*, vol. 30, no. 4, Mar. 2021, Art. no. 041403.

[32] G. Toderici et al., "Variable rate image compression with recurrent neural networks," 2015, *arXiv:1511.06085*.

[33] K. Islam, L. M. Dang, S. Lee, and H. Moon, "Image compression with recurrent neural network and generalized divisive normalization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2021, pp. 1875–1879.

[34] A. Van den Oord et al., "Conditional image generation with PixelCNN decoders," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1–9.

[35] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[36] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.

[37] D. Valsesia and E. Magli, "High-throughput onboard hyperspectral image compression with ground-based CNN reconstruction," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 12, pp. 9544–9553, Dec. 2019.

[38] J. Lei Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.

[39] Z. Yang, Z. Dai, R. Salakhutdinov, and W. W. Cohen, "Breaking the softmax bottleneck: A high-rank RNN language model," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–18.

[40] M. H. P. Fuchs and B. Demir, "HySpecNet-11k: A large-scale hyperspectral dataset for benchmarking learning-based hyperspectral image compression methods," 2023, *arXiv:2306.00385*.

[41] U-Houston. (2018). *Data Fusion Contest (DFC) 2018 Dataset*. Accessed: Jun. 25, 2024. [Online]. Available: https://hyperspectral.ee.uh.edu/?page_id=1075

[42] S. Cogliati et al., "The PRISMA imaging spectroscopy mission: Overview and first performance analysis," *Remote Sens. Environ.*, vol. 262, Sep. 2021, Art. no. 112499.

[43] Z. Wang, E. P. Simoncelli, and A. C. Bovik, "Multiscale structural similarity for image quality assessment," in *Proc. 37th Asilomar Conf. Signals, Syst. Comput.*, vol. 2, Nov. 2003, pp. 1398–1402.

[44] F. A. Kruse et al., "The spectral image processing system (SIPS)-interactive visualization and analysis of imaging spectrometer data," *Remote Sens. Environ.*, vol. 44, nos. 2–3, pp. 145–163, May 1993. [Online]. Available: https://www.sciencedirect.com/science/article/pii/003442579390013N

Diego Valsesia (Member, IEEE) received the Ph.D. degree in electronic and communication engineering from the Politecnico di Torino, Turin, Italy, in 2016.

He is currently an Assistant Professor with the Department of Electronics and Telecommunications (DET), Politecnico di Torino. His main research interests include the processing of remote sensing images and deep learning for inverse problems in imaging.

Dr. Valsesia is a member of the EURASIP Technical Area Committee for Signal and Data Analytics for Machine Learning and the ELLIS Society. He was a recipient of the IEEE ICIP 2019 Best Paper Award and the IEEE Multimedia 2019 Best Paper Award. He is an Associate Editor of the IEEE TRANSACTIONS ON IMAGE PROCESSING, for which he received the 2023 Outstanding Editorial Board Member Award.

Tiziano Bianchi (Member, IEEE) received the M.Sc. degree (Laurea) in electronic engineering and the Ph.D. degree in information and telecommunication engineering from the University of Florence, Florence, Italy, in 2001 and 2005, respectively.

From 2005 to 2012, he was a Research Assistant with the Department of Electronics and Telecommunications, University of Florence. In 2012, he joined the Politecnico di Torino, Turin, Italy, as an Assistant Professor, where he is currently an Associate Professor. He has authored over 100 papers in international journals and conference proceedings. His research interests include multimedia security technologies, multimedia forensics, and the processing of remote sensing images.

Dr. Bianchi was a recipient of the IEEE Multimedia 2019 Best Paper Award and the 2021 and 2022 Best Associate Editor Award of the *Journal of Visual Communication and Image Representation*. He is currently an Associate Editor of the IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY and a Senior Area Editor of the *Journal of Visual Communication and Image Representation*.

Enrico Magli (Fellow, IEEE) received the M.Sc. and Ph.D. degrees from the Politecnico di Torino, Turin, Italy, in 1997 and 2001, respectively.

He is currently a Full Professor with the Politecnico di Torino, where he also leads the Image Processing and Learning Group, performing research in the fields of deep learning for image and video processing, image compression and image forensics for multimedia, and remote sensing applications.

Dr. Magli is a fellow of the ELLIS Society for Advancement of Artificial Intelligence in Europe and was an IEEE Distinguished Lecturer from 2015 to 2016. He was a recipient of the IEEE Geoscience and Remote Sensing Society 2011 Transactions Prize Paper Award, the IEEE ICIP 2015 Best Student Paper Award (as a senior author), the IEEE ICIP 2019 Best Paper Award, the IEEE Multimedia 2019 Best Paper Award, and the 2010 and 2014 Best Associate Editor Awards of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY. He is an Associate Editor of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY and the *EURASIP Journal on Image and Video Processing*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 12:08:51 UTC from IEEE Xplore. Restrictions apply.